



ARQUITETURA DE SISTEMAS

AULA 8 - MICROKERNEL E ORIENTADA A SERVIÇOS



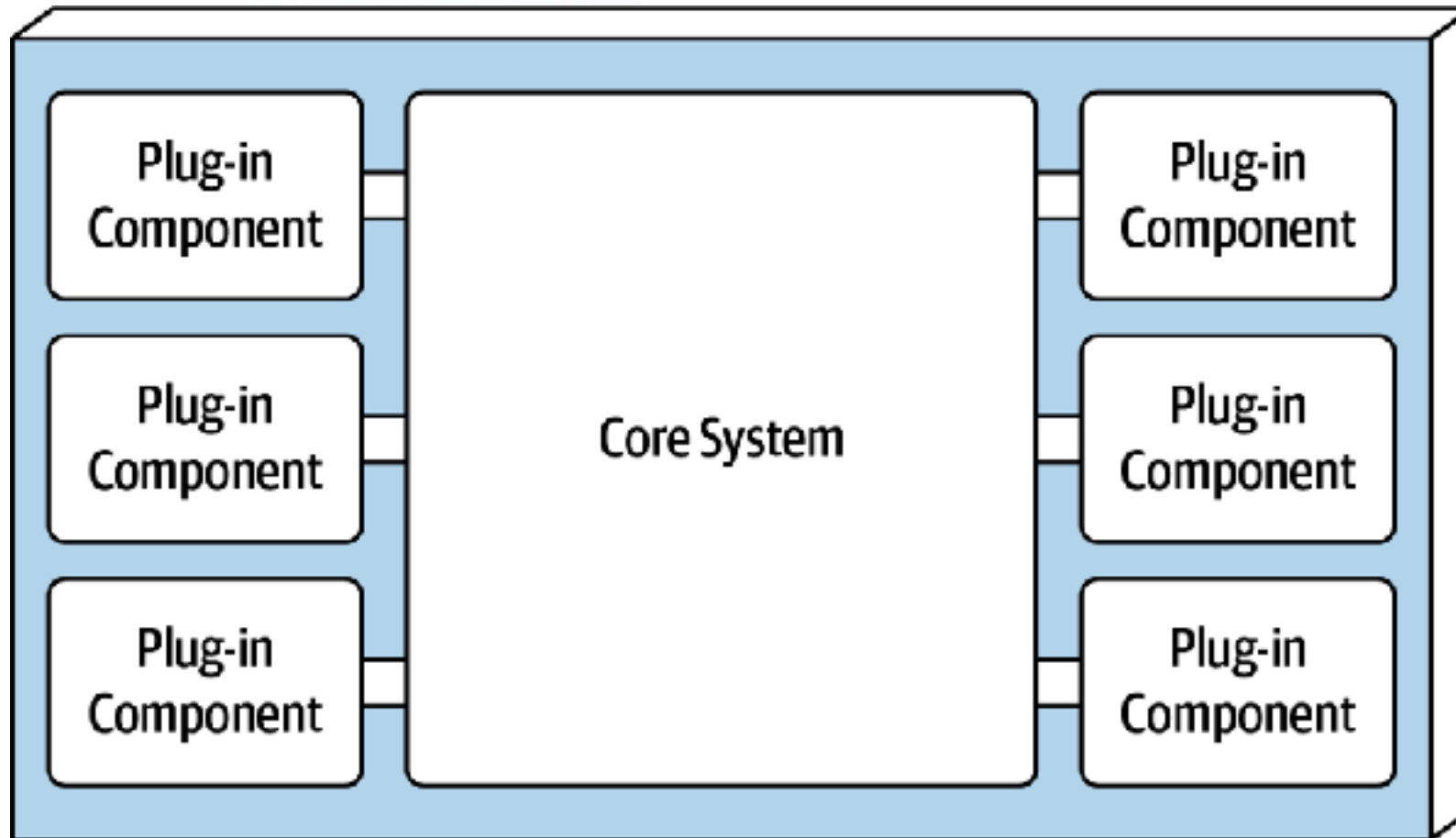
ARQUITETURA MICROKERNEL

INTRODUÇÃO AO MICROKERNEL

- ❑ Conceito: Arquitetura Microkernel (também conhecida como plug-in architecture) é dividido em duas partes: uma com um núcleo mínimo de funcionalidades, e outra que são os módulos de extensão plugáveis. Esses módulos adicionam recursos adicionais sem modificar o núcleo.
- ❑ Características Gerais: Enfatiza extensibilidade e adaptabilidade. Novas funcionalidades são adicionadas como plug-ins independentes, mantendo o núcleo estável e isolado de mudanças frequentes.
- ❑ Exemplo mais famoso: Linux



TOPOLOGIA BÁSICA DO MICROKERNEL



Topologia básica do estilo Microkernel. O sistema possui um núcleo central (Core System) cercado por componentes de plug-in que estendem suas funcionalidades. O núcleo comunica-se com os plug-ins por interfaces bem definidas, e cada plug-in adiciona comportamentos especializados sem alterar o núcleo.

TOPOLOGIA BÁSICA DO MICROKERNEL

- ❑ Core System (Núcleo): Componente central monolítico que contém apenas as funcionalidades mínimas necessárias para executar a aplicação. Todos os serviços básicos residem aqui.
- ❑ Plug-in Components: Módulos externos que se conectam ao núcleo para oferecer funcionalidades extras ou customizadas. Podem ser adicionados ou removidos conforme a necessidade, sem afetar o núcleo ou outros plug-ins.
- ❑ Isolamento de Lógica: A lógica de negócio é dividida entre o núcleo e os plug-ins. Funcionalidades voláteis ou específicas ficam nos plug-ins, enquanto o fluxo principal e dados centrais permanecem no núcleo, garantindo estabilidade.



CORE SYSTEM – O NÚCLEO DO MICROKERNEL

- ❑ Função do Núcleo: Provê as operações básicas e infraestrutura para rodar o sistema. Sem plug-ins, ele oferece apenas o caminho funcional “padrão” pela aplicação (ex.: em um IDE, o núcleo seria o editor de texto simples capaz de abrir, editar e salvar arquivos).
- ❑ Simplicidade Proposital: Mantém apenas lógica essencial, delegando complexidades e variações aos plug-ins. Isso torna o núcleo mais extensível, manutenível e testável, pois ele não é inchado com comportamentos especiais.
- ❑ Projeto Interno: Pode ser implementado seguindo uma arquitetura em camadas (layered) ou um monólito modular. Em sistemas maiores, o núcleo pode até ser reparticionado em serviços de domínio separados, cada um com seus próprios plug-ins, embora permaneça a ideia de um kernel central por domínio.



PLUG-IN COMPONENTS – MÓDULOS DE EXTENSÃO

- ❑ Papel dos Plug-ins: São componentes independentes que adicionam funcionalidades especializadas ou recursos customizados ao sistema. Cada plug-in geralmente implementa uma funcionalidade isolada (por exemplo, um método de pagamento específico, um formato de relatório, um plugin de vídeo em um navegador).
- ❑ Independência: Foram concebidos para não terem dependências fortes uns com os outros. Cada plug-in interage com o núcleo através de interfaces, evitando acoplamento direto entre plug-ins. Isso permite adicionar, remover ou modificar plug-ins sem impactar os demais.



PLUG-IN COMPONENTS – MÓDULOS DE EXTENSÃO

- ❑ Tempo de Execução vs Compilação: Plug-ins de runtime podem ser instalados/ativados dinamicamente, com o sistema rodando, sem necessidade de reiniciar ou reimplantar a aplicação (útil para extensões em aplicativos como IDEs ou browsers). Já plug-ins de compilação (estáticos) são ligados ao sistema em build/deploy: são mais simples de gerenciar porém qualquer mudança exige recompilação e redistribuição do sistema inteiro.
- ❑ Dados Próprios: Um plug-in pode ter seu próprio armazenamento de dados local para informações específicas de sua funcionalidade. Alterações no banco de dados global afetam apenas o núcleo; cada plug-in que precisar persistência pode usar tabelas ou arquivos próprios, isolando dados específicos e evitando impacto no núcleo ou em outros plug-ins.



INTEGRAÇÃO E COMUNICAÇÃO NO MICROKERNEL

- ❑ Comunicação Núcleo->Plug-in: Geralmente ponto-a-ponto via chamadas de método ou função. O núcleo expõe interfaces ou serviços que os plug-ins implementam; para usar um plug-in, o núcleo invoca o método de entrada desse plug-in (como se fosse “plugar” e “desplugar” funcionalidades).
- ❑ Registro de Plug-ins: O núcleo mantém um registry dos plug-ins disponíveis, com informações como nome do plug-in, interface de comunicação, dados que espera/retorna e protocolo (se remoto). Esse registro pode ser uma simples estrutura interna (mapa chave->instância) ou um serviço de descoberta mais elaborado. Ele permite ao núcleo saber quais plug-ins estão ativos e como chamá-los quando necessário.



INTEGRAÇÃO E COMUNICAÇÃO NO MICROKERNEL

- ❑ Contratos: Núcleo e plug-ins acordam contratos de interface: definições de métodos, formatos de dados de entrada/saída (por exemplo, classes, XML ou JSON) que ambos entendem. Todos os plug-ins de um certo tipo seguem o mesmo contrato. Se um plug-in de terceiro tiver um contrato diferente, costuma-se escrever um adaptador entre o contrato externo e o contrato padrão interno, para que o núcleo não precise tratar cada plug-in de forma especial.
- ❑ Acesso Remoto (variação): Embora menos comum, é possível invocar plug-ins como serviços remotos (por exemplo via chamadas REST ou mensageria) em vez de chamados locais. Isso tornaria a arquitetura distribuída, permitindo cada plug-in rodar em processo ou servidor separado. Cuidado: essa abordagem aumenta muito a complexidade (rede, latência, deploy de múltiplos componentes) e transforma o microkernel em um sistema distribuído, perdendo parte da simplicidade original. Ainda assim, o núcleo continua sendo ponto de entrada central, ou seja, toda requisição passa pelo core antes de alcançar o serviço plug-in remoto.



VANTAGENS DO ESTILO MICROKERNEL

- ❑ Extensibilidade: Facilita adicionar novas funcionalidades ou variações sem alterar o core. Basta desenvolver um novo plug-in conforme os contratos e registrá-lo. Ideal para produtos que precisam ser customizados para diferentes clientes ou evoluir ao longo do tempo com mínimos impactos.
- ❑ Isolamento de Mudanças: Código propenso a mudanças frequentes (regras de negócio voláteis, funcionalidades experimentais) fica isolado nos plug-ins. Assim, atualizações ou correções nesses módulos não requerem reimplantar todo o sistema e têm baixo risco de quebrar funcionalidades centrais.



VANTAGENS DO ESTILO MICROKERNEL

- ❑ Manutenibilidade e Testabilidade: Separar o núcleo das extensões torna cada parte menor e mais focada. O núcleo permanece simples, facilitando testes unitários e manutenção, enquanto cada plug-in pode ser desenvolvido e testado isoladamente. Bugs em um plug-in não comprometem o núcleo nem outros plug-ins.
- ❑ Adaptabilidade: O mesmo produto pode ser adaptado a diferentes contextos ativando ou desativando certos plug-ins. Por exemplo, uma mesma plataforma pode atender clientes de portes ou segmentos distintos apenas mudando o conjunto de módulos ativos, sem precisar ramificar o código base do núcleo.



DESVANTAGENS DO ESTILO MICROKERNEL

- ❑ Complexidade Adicional: Introduzir mecanismos de plug-in e contratos adiciona complexidade ao design. É um peso extra de arquitetura que só compensa se a necessidade de extensibilidade for real. Para aplicações simples, um monólito tradicional poderia ser mais fácil de construir e manter.
- ❑ Performance e Overhead: O direcionamento de chamadas através do núcleo e a carga/descarga dinâmica de módulos podem introduzir overhead. Se mal projetado, a comunicação extra e a abstração de interfaces podem impactar a performance em comparação a chamadas diretas dentro de um monólito integrado.



DESVANTAGENS DO ESTILO MICROKERNEL

- ❑ Ponto Único Central: O core permanece como ponto crítico: se ele falhar, todo o sistema para. Além disso, ele pode se tornar um gargalo se muitos plug-ins forem acionados simultaneamente, pois toda interação passa por ele. Escalar a aplicação geralmente implica escalar o núcleo inteiro junto com plug-ins, já que não são sistemas independentes.
- ❑ Gerenciamento de Versões e Compatibilidade: Manter compatibilidade entre o núcleo e múltiplos plug-ins, possivelmente desenvolvidos por times diferentes ou terceiros, pode ser desafiador. Mudanças no contrato do núcleo exigem atualizar vários plug-ins. Cada plug-in pode ter sua versão, e garantir que todos funcionem em conjunto requer processos rigorosos de gestão de versões.



EXEMPLOS E CASOS DE USO DO MICROKERNEL

- ❑ IDEs e Ferramentas de Desenvolvimento: Ambientes como Eclipse e Visual Studio Code adotam microkernel: oferecem um núcleo editor básico e todas as outras linguagens/ferramentas são plug-ins. Isso permite uma IDE altamente customizável, onde funcionalidades (suporte a Java, Python, controle de versão, etc.) podem ser instaladas ou atualizadas independentemente.
- ❑ Navegadores Web: Google Chrome, Firefox e outros navegadores modernos têm núcleos de renderização e funcionalidades mínimas, enquanto extensões e add-ons funcionam como plug-ins. Usuários podem adicionar bloqueadores de script, integradores de notas, temas, etc., sem alterar o navegador em si.



EXEMPLOS E CASOS DE USO DO MICROKERNEL

- ❑ Aplicações Corporativas Extensíveis: Produtos como Jira ou Confluence (da Atlassian) fornecem um core de gestão de projetos/ conteúdo e permitem plug-ins para adaptar a ferramenta a diferentes processos (workflow customizado, integrações específicas). Isso aumenta o valor do produto para vários cenários sem ramificar o core.
- ❑ Sistema de Regras de Negócio (Exemplo Seguradoras): Considere uma seguradora que processa sinistros (claims) com regras diferentes por jurisdição. Usando microkernel, o fluxo padrão de processamento (registro, avaliação básica, pagamento) fica no núcleo e regras específicas de cada estado/país são implementadas em plug-ins separados. Cada plug-in contém as regras de negócio e validações daquela região. Assim, adicionar ou mudar regras regionais não exige tocar no núcleo ou interromper outras regiões: basta atualizar o módulo correspondente. O núcleo permanece estável e raramente muda, enquanto a complexidade regulatória fica encapsulada nos plug-ins, tornando o sistema mais flexível e fácil de manter.



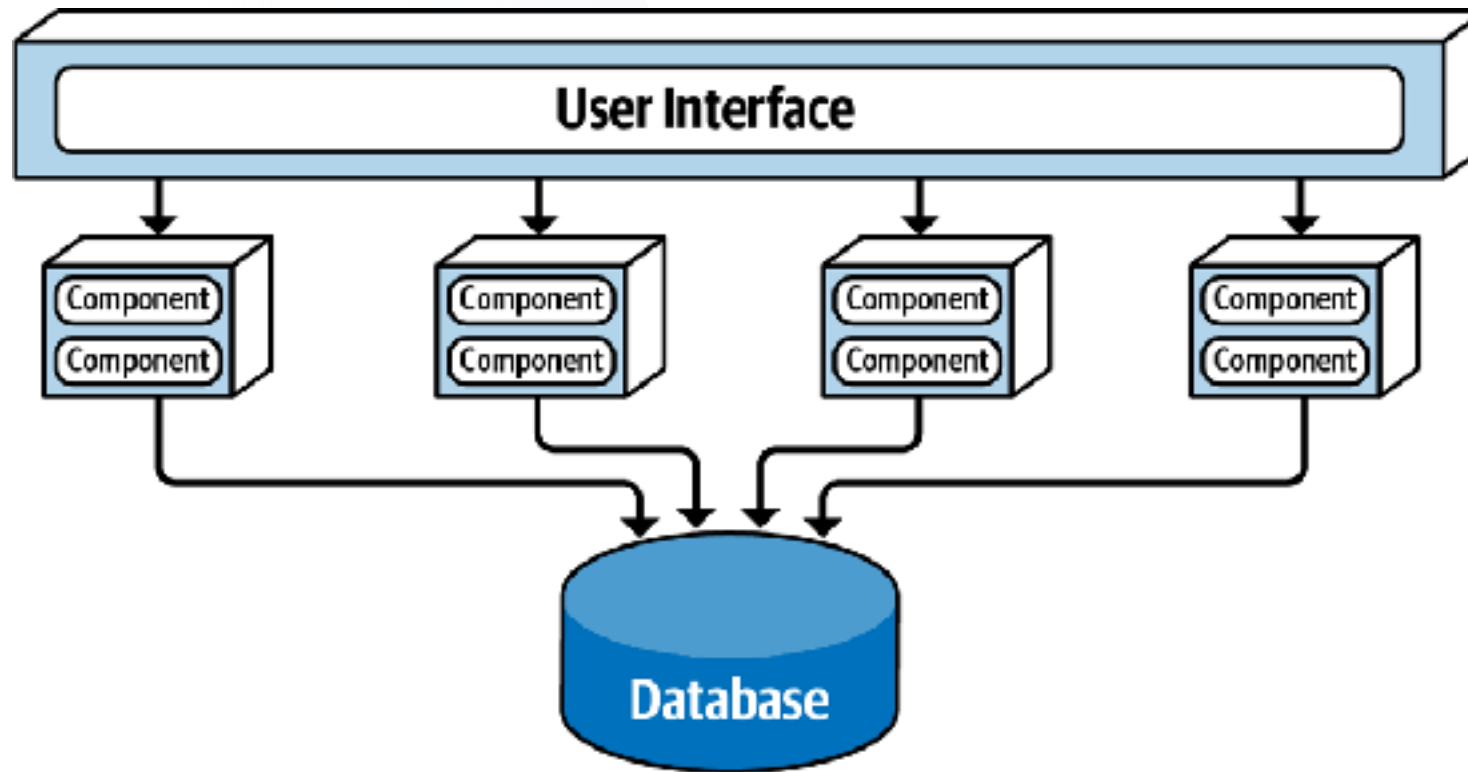
ARQUITETURA ORIENTADA A SERVIÇOS

INTRODUÇÃO AO SERVICE-BASED

- ❑ Conceito: A arquitetura orientada a serviços (Service-Based) é um estilo híbrido que combina elementos de monólitos e microserviços. Surge como uma abordagem pragmática para distribuir uma aplicação em serviços sem atingir o nível de granularidade e complexidade dos microservices completos.
- ❑ Contexto: Muito utilizada em aplicações de negócios que precisam ser modulares, mas evitam o alto custo e esforço de um microservice para cada funcionalidade. Oferece um equilíbrio entre flexibilidade arquitetural e simplicidade operacional.
- ❑ Comparativo: Embora seja uma arquitetura distribuída (vários serviços), o service-based mantém partes compartilhadas (como banco de dados único), resultando em menos complexidade e menor custo comparado a arquiteturas distribuídas puras (microservices ou event-driven). É uma escolha popular quando microservices “puros” seriam exagero.



TOPOLOGIA BÁSICA DO SERVICE-BASED



Topologia básica do estilo Service-Based. A interface de usuário comunica-se com vários serviços de domínio independentes, e todos os serviços compartilham um único banco de dados monolítico. Nesse arranjo, cada serviço cuida de um subconjunto funcional da aplicação (por exemplo, Pedido, Pagamento, Catálogo), proporcionando modularidade sem exigir bancos separados. A base de dados comum simplifica a consistência dos dados, mas cria um ponto central de dependência.

TOPOLOGIA BÁSICA DO SERVICE-BASED

- ❑ Interface de Usuário Separada: A camada de apresentação (UI) é um componente isolado, implantado separadamente. Ela não está acoplada diretamente à lógica de negócio, comunicando-se com os serviços via chamadas remotas (ex: requisições REST).
- ❑ Serviços de Domínio: Funcionalidades de negócio são divididas em serviços independentes, cada um correspondendo a um domínio ou módulo da aplicação (por exemplo, Serviço de Clientes, Serviço de Pedidos, Serviço de Pagamentos). Cada serviço é implantado separadamente, como se fosse um pequeno monólito próprio, com sua camada de negócio e acesso a dados.
- ❑ Banco de Dados Único: Diferente dos microservices, aqui todos os serviços compartilham um mesmo banco de dados monolítico centralizado. Isso significa que os dados de todos os domínios ficam juntos, permitindo consultas e joins diretos entre domínios se necessário. Em contrapartida, introduz forte acoplamento a nível de dados: alterações no esquema afetam potencialmente todos os serviços.



DOMAIN SERVICES

- ❑ Coarse-Grained Services: Cada domínio é implementado como um serviço coeso porém abrangente. Em vez de dezenas de microserviços pequenos, uma arquitetura service-based costuma ter poucos serviços. Cada um abrange um conjunto de funcionalidades relacionadas. Por exemplo, um OrderService trata desde criar pedidos, calcular total, até atualizar estoque (tudo dentro do mesmo serviço).
- ❑ Independência de Implantação: Cada serviço de domínio é desenvolvido e implantado como uma unidade separada. Não há necessidade de containerização obrigatória: pode ser implantado em servidores de aplicação tradicionais ou em containers Docker, conforme a necessidade. O importante é que cada serviço pode ser implantado/escalado individualmente (embora todos dependam do mesmo banco).
- ❑ Módulos Internos: Dentro de cada serviço, geralmente existe uma mini-arquitetura em camadas: por exemplo, um API Facade que recebe as requisições do UI, uma camada de lógica de negócio e uma camada de persistência que interage com o banco. Em alguns casos, um serviço de domínio grande pode ser subdividido internamente em módulos menores (subdomínios), semelhante a um monólito modular, mas permanece como um único serviço implantável.



COMUNICAÇÃO E CAMADA DE API

- ❑ Chamadas Remotas: A interação entre a UI e os serviços é feita por mecanismos de comunicação remota. O mais comum é REST sobre HTTP, devido à simplicidade e leveza. Alternativamente podem ser usados protocolos de mensageria, gRPC/RPC ou até SOAP, dependendo do contexto e necessidade de integração.
- ❑ Acesso Direto vs Gateway: Na forma mais simples, a UI faz chamadas diretas a cada serviço de domínio relevante (por exemplo, chamadas REST do front-end web para os endpoints de cada serviço). Em cenários mais complexos, pode-se adicionar uma camada de API Gateway/Proxy entre a UI e os serviços: esse gateway unifica chamadas, roteia para os serviços apropriados e pode tratar responsabilidades transversais (autenticação, logs, monitoramento). Entretanto, para uma aplicação interna, muitas vezes a UI se comunica diretamente com os serviços sem gateway intermediário.



COMUNICAÇÃO E CAMADA DE API

- ❑ Instâncias e Load Balancing: Normalmente existe uma instância de cada serviço de domínio (cada um já sendo capaz de atender múltiplas requisições). Se for necessário suportar mais usuários ou garantir tolerância a falhas, pode-se subir múltiplas instâncias de um mesmo serviço atrás de um balanceador de carga. Assim, a UI ou o gateway envia requisições para instâncias redundantes, garantindo escalabilidade horizontal quando preciso.



VARIANTES DE TOPOLOGIA NO SERVICE-BASED

- ❑ Interfaces Dedicadas por Domínio: Uma variação possível é segmentar também a interface de usuário por domínio. Por exemplo, aplicações muito grandes podem ter diferentes front-ends especializados (um front-end para clientes, outro para admins, etc.), cada qual comunicando com serviços específicos. Essa abordagem lembra micro frontends e pode melhorar a autonomia de equipes por domínio, embora complique um pouco a experiência unificada do usuário.
- ❑ Múltiplos Bancos de Dados: Apesar do típico ser um banco único, pode-se optar por separar o banco por serviço (cada serviço de domínio com seu esquema ou banco próprio). Isso reduz o acoplamento de dados: cada serviço gerencia suas tabelas independentemente, parecido com microservices. Atenção: ao fazer isso, deve-se garantir que um serviço não precise de dados do banco de outro, pois a arquitetura service-based evita comunicações entre serviços. Se os dados ficarem fragmentados e um serviço precisar do outro, acaba-se reintroduzindo dependências complexas.



VARIANTES DE TOPOLOGIA NO SERVICE-BASED

- ❑ Camada de API Externa: Quando é necessário expor funcionalidades para fora (por exemplo, disponibilizar serviços para consumo por parceiros ou integrar com outros sistemas), pode-se incluir um API Gateway na frente dos serviços. Esse gateway consolida as APIs dos vários serviços em um ponto único de acesso externo e pode mover para fora da UI certas responsabilidades transversais (logging centralizado, segurança, auditoria, descoberta de serviço). Essa camada adicional melhora a segurança e manutenibilidade em integrações externas, mas adiciona outro componente para gerenciar.



GRANULARIDADE E ORQUESTRAÇÃO DE SERVIÇOS

- ❑ **Serviços Coesos vs Microserviços:** Em service-based, cada serviço orquestra internamente várias funções de negócio do seu domínio. A orquestração é interna, dentro do processo do próprio serviço (por exemplo, o OrderService executa sequencialmente as etapas de validar pedido, processar pagamento e atualizar estoque). Já em microservices puros, essas etapas poderiam estar em serviços separados (Pedido, Pagamento, Estoque) exigindo orquestração entre serviços remotos (muito mais complexa).
- ❑ **Transações ACID:** Como todos os módulos compartilham o mesmo banco, cada serviço pode usar transações ACID tradicionais garantindo consistência forte dentro do escopo do serviço/domínio. No exemplo do OrderService, todas as operações de pedido e estoque podem ser confirmadas em uma única transação de banco de dados, assegurando atomicidade e isolamento.



GRANULARIDADE E ORQUESTRAÇÃO DE SERVIÇOS

- ❑ Consistência vs Disponibilidade: Em microservices com bancos separados, não se pode ter transações ACID abrangendo múltiplos serviços facilmente; adota-se o princípio BASE (Basic Availability, Soft state, Eventual consistency), aceitando inconsistência temporária e coordenação assíncrona para manter disponibilidade. No service-based, por usar um banco unificado, não há necessidade de eventual consistency entre serviços: a consistência é imediata dentro de cada serviço, simplificando o modelo de dados e evitando a necessidade de mecanismos de compensação ou sagas complexos.
- ❑ Impacto de Mudanças: A granularidade mais grossa implica que uma mudança de lógica dentro de um serviço requer o redeploy daquele serviço inteiro (que cobre muitas funcionalidades). Comparativamente, em microservices uma mudança afetaria apenas um microserviço pequeno. Assim, service-based sacrifica um pouco de independência de implantação em troca de um design menos fragmentado. Ainda assim, por terem poucos serviços, o impacto de mudanças costuma ser controlável e os times mantêm a responsabilidade dentro do limite de cada domínio.



VANTAGENS DO ESTILO SERVICE-BASED

- ❑ Reaproveitamento de Consistência: O banco unificado permite integridade de dados imediata: não é preciso implementar mecanismos de sincronização entre serviços. Regras de negócio que envolvem múltiplos domínios podem muitas vezes ser tratadas com consultas SQL e transações únicas, aproveitando joins e constraints do SGBD tradicional.
- ❑ Evolução Gradual: Uma arquitetura service-based pode servir como etapa intermediária entre um monólito e uma arquitetura de microservices plena. Ela modulariza a aplicação em componentes separáveis, o que no futuro facilita extrair serviços adicionais ou mesmo migrar para microservices se necessário. É uma abordagem pragmática para começar a dividir um sistema grande sem refatoração extrema de uma vez.



DESAFIOS E DESVANTAGENS DO SERVICE-BASED

- ❑ Acoplamento de Banco de Dados: Alterações no esquema do banco (ex: modificar uma tabela compartilhada) podem demandar atualizar múltiplos serviços e coordenar deploys para evitar incompatibilidades. Cada serviço depende do modelo de dados global, o que requer coordenação cuidadosa entre equipes para evoluir o banco sem interromper a aplicação.
- ❑ Limites Ambíguos: Decidir a divisão ótima dos domínios pode ser difícil. Se os serviços forem muito grandes, pode se aproximar de um monólito distribuído que não resolve isolamento de contexto. Se forem muito pequenos, entra-se no terreno dos micros serviços e seus desafios. Encontrar o tamanho “certo” do serviço é crucial e requer bom entendimento do domínio de negócio (conceito de bounded context ajuda nessa definição).



DESAFIOS E DESVANTAGENS DO SERVICE-BASED

- ❑ Escalabilidade Limitada pelo BD: Embora cada serviço possa escalar horizontalmente em instâncias, todos continuam a competir por I/O no mesmo banco de dados. O banco centralizado pode se tornar gargalo de performance conforme aumenta a carga. Diferente de microservices onde cada serviço pode escalar junto com seu banco, aqui o database é único: escalabilidade de banco requer soluções de clusterização ou replicas que adicionam complexidade.
- ❑ Risco de Comunicação Oculta: Deve-se evitar ao máximo que um serviço chame outro serviço diretamente (a filosofia service-based é os serviços só se comunicarem via o banco comum). Se times começarem a introduzir integrações diretas entre serviços, a arquitetura se degrada para um híbrido acoplado e perde os benefícios de simplicidade. Portanto, é importante delinear bem os contextos: cada serviço opera de forma autônoma no seu domínio, interagindo com outros apenas por meio dos dados compartilhados, não via chamadas diretas.



EXEMPLO DE APLICAÇÃO SERVICE-BASED

- ❑ E-commerce Modular: Imagine uma aplicação de comércio eletrônico dividida em serviços de domínio: Cadastro de Usuários, Catálogo de Produtos, Pedidos, Pagamentos e Entregas. Cada serviço é independente. Por exemplo, o serviço de Pedidos lida com criar e gerenciar pedidos, o de Pagamentos processa transações, etc. Todos compartilham um único banco de dados central que contém tabelas de usuários, produtos, pedidos, pagamentos, etc.
- ❑ Funcionamento: O front-end (UI) da loja faz requisições REST para cada serviço conforme necessário (uma página de checkout pode chamar o serviço de Pedidos para criar o pedido e o de Pagamentos para efetuar a cobrança). Como o banco é comum, o serviço de Pedidos pode gravar um novo pedido e o serviço de Pagamentos gravar uma transação de pagamento, ambos visíveis em uma única base: permitindo, por exemplo, uma consulta que junte dados de pedidos e pagamentos para exibir ao usuário.



EXEMPLO DE APLICAÇÃO SERVICE-BASED

- ❑ Benefícios no Exemplo: A empresa consegue desenvolver módulos separadamente (diferentes times podem focar em Pagamentos vs. Catálogo, por exemplo). A integridade dos dados é mantida facilmente pois tudo está no mesmo banco (um pedido e seu pagamento permanecem consistentes juntos em uma transação). Não há a sobrecarga de gerenciar 50 microserviços: apenas alguns serviços maiores. Se no futuro a parte de Pagamentos exigir requisitos de escala ou isolamento de dados muito específicos, pode-se então extrair esse serviço para um microservice independente com banco próprio, sem ter que refatorar todo o sistema de uma vez.
- ❑ Trade-Off com Microservices: Nesse cenário service-based, a aplicação ganha modularidade e certo isolamento de falhas (se o serviço de Entregas cair, os demais continuam funcionais). Porém, como todos dependem do banco central, uma indisponibilidade do banco pararia todo o sistema: trade-off consciente para evitar a complexidade de replicar dados entre serviços, o que seria necessário em microservices. Para muitas aplicações de médio porte, essa compensação é aceitável em prol de uma solução mais simples de implementar e manter.



ATÉ A PRÓXIMA AULA



@eldermoraes